
Bitboard version of Tetris AI

Xingguo Chen, Pingshou Xiong, Zhenyu Luo, Mengfei Hu, Xinwen Li, Yongzhou Lü, Guang Yang, Chao Li, Shangdong Yang

Abstract

The efficiency of game engines and policy optimization algorithms is crucial for training reinforcement learning (RL) agents in complex sequential decision-making tasks, such as Tetris. Existing Tetris implementations suffer from low simulation speeds, suboptimal state evaluation, and inefficient training paradigms, limiting their utility for large-scale RL research. To address these limitations, this paper proposes a high-performance Tetris AI framework based on bitboard optimization and improved RL algorithms. First, we redesign the Tetris game board and tetrominoes using bitboard representations, leveraging bitwise operations to accelerate core processes (e.g., collision detection, line clearing, and Dellacherie-Thiery Features extraction) and achieve a 53-fold speedup compared to OpenAI Gym-Tetris. Second, we introduce an afterstate-evaluating actor network that simplifies state value estimation by leveraging Tetris’s afterstate property, outperforming traditional action-value networks with fewer parameters. Third, we propose a buffer-optimized Proximal Policy Optimization (PPO) algorithm that balances sampling and update efficiency, achieving an average score of 3,829 on 10×10 grids within 3 minutes. Additionally, we develop a Python-Java interface compliant with the OpenAI Gym standard, enabling seamless integration with modern RL frameworks. Experimental results demonstrate that our framework enhances Tetris’s utility as an RL benchmark by bridging low-level bitboard optimizations with high-level AI strategies, providing a sample-efficient and computationally lightweight solution for scalable sequential decision-making research.

Keywords

Reinforcement learning; Bitboard; Tetris; DT Features; Proximal Policy Optimization; Afterstate

Introduction

Tetris, a classic sequential decision-making game invented by Alexey Pajitnov in 1984, has long served as a foundational benchmark for reinforcement learning (RL), approximate dynamic programming, and optimization algorithms—owing to its unique combination of challenges: an NP-hard optimal strategy problem¹, an enormous state space (approximately 7×2^{200} configurations for 10×20 grids²), and the inevitability of game termination under adversarial piece sequences (e.g., prolonged Z/S pieces)^{3,4}. Its status is reinforced by its prominent role in the 2008-2009 Reinforcement Learning Competitions of ICML workshops⁵, collectively solidifying Tetris’s value for testing RL agents’ ability to handle uncertainty and long-term planning.

Subsequent Tetris AI research, systematically summarized in Table 1, compares representative methods across key metrics: average removed lines, training samples, feature sets, and piece generators. Early approaches like BCTS² based on noise cross entropy¹⁰, achieved 3,000 average scores but required 6.5×10^7 training samples—an order of magnitude more than later methods such as CBMPI^{6,7} (4,300 avg. with 8×10^6 samples) and dSiLU-TD(λ)⁸ (4,900 avg., the highest in the table, with 200,000 samples). Even STEW⁹ (4,800 avg.) relied on a 7-Bag generator (a simplified deterministic setting) to avoid overfitting, sacrificing generalization to classic Tetris’s true stochasticity (i.i.d. piece drawing). While Table 1 confirms Tetris’s ability to distinguish method performance across sample efficiency and feature robustness, it also exposes a critical

gap: high performance often demands massive training resources, while existing open-source implementations (e.g., OpenAI Gym-Tetris¹¹) suffer from severe runtime inefficiency—taking 12.92 seconds for 10,000 samples vs. 0.24 seconds for our optimized design—hindering large-scale RL training.

Beyond the ‘performance-efficiency tradeoff’ highlighted by Table 1, existing Tetris research faces unresolved limitations that restrict its utility for scalable RL.

- *Engine inefficiency* plagues grid-based implementations: traditional representations fail to leverage bitwise operations for fast collision detection, line clearing, and state evaluation—core processes that our bitboard design accelerates.
- *Policy optimization is constrained*: state-of-the-art methods either depend on complex hand-crafted features (e.g., Bertsekas features¹² in dSiLU-TD(λ)⁸) or trajectory-based training paradigms that waste resources on low-quality early-stage samples.

These issues collectively limit Tetris’s potential to drive progress in sequential decision-making research.

To address these gaps, this paper proposes a high-efficiency Tetris AI framework, with core contributions aligned to the key innovations of the study:

Corresponding author:

Xingguo Chen, School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, Jiangsu, 210023, China
Email: chenxg@njupt.edu.cn

Table 1. Records in 10×10 Tetris AI.

Method	Avg. Removed Lines	Training Samples	Feature	Generator
BCTS ²	3,000	6.5×10^7	DT features	Random
CBMPI ^{6,7}	4,300	8×10^6	DT features + RBF heights	Random
dSiLU-TD(λ) ⁸	4,900	200,000	Bertsekas features + neural networks	Random
STEW ⁹	4,800	238,000	DT features	7-Bag random
Buffered PPO (Ours)	3,850	61,440	DT features	Random

- *Bitboard-Based Tetris Implementation:* Redesigning the game board (10 columns as 32-bit integers) and tetrominoes using bitwise operations, achieving a 53-fold speedup over OpenAI Gym-Tetris. This acceleration enables efficient simulation for large-scale RL training by optimizing collision detection, line clearing, and Dellacherie-Thierry (DT) feature calculation.
- *Afterstate-Evaluating Actor + Buffer-Optimized PPO:* The afterstate-based Actor simplifies state value estimation by leveraging Tetris’s afterstate property (board configuration post-action but pre-next-piece), outperforming traditional action-value networks. The buffer-optimized PPO reduces training steps to 61,440 (1/1058 of BCTS, 1/3 of dSiLU-TD(λ)) while maintaining a competitive average score of 3,829 on 10×10 grids.
- *OpenAI Gym-Compliant Python-Java Interface:* Built via Jpype¹³, this interface bridges Java’s bitwise performance with Python’s RL framework compatibility (e.g., PyTorch, TensorFlow), enabling rapid algorithm prototyping.

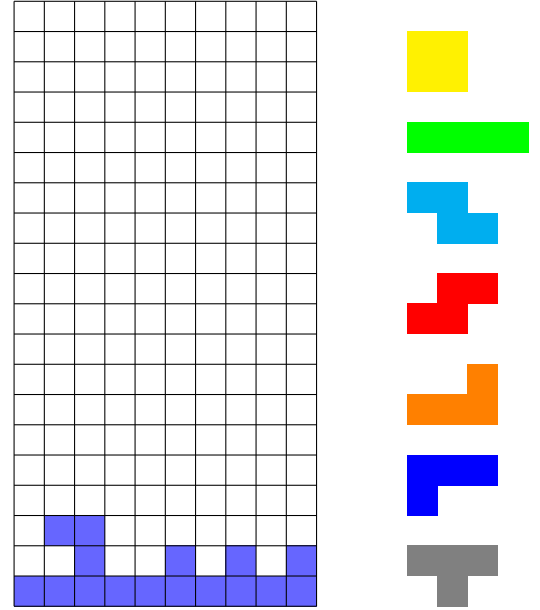
The rest of this paper is organized as follows: Section 2 covers Tetris background (game mechanics, MDP formalization, DT features). Section 3 details the bitboard implementation and accelerated operations. Section 4 presents afterstate Actor and the buffer-based PPO framework. Section 5 validates performance on $10 \times 10/10 \times 20$ grids, comparing to state-of-the-art methods. Section 6 gives the conclusion.

Background

In this background section, we provide foundational knowledge on the Tetris game and its application in artificial intelligence, emphasizing reinforcement learning frameworks. We first describe the core mechanics of Tetris, including piece manipulation, line clearing, and the critical concept of afterstates, which facilitate efficient state evaluation. Subsequently, we formalize Tetris as a Markov Decision Process (MDP), detailing state and action representations through feature engineering like Dellacherie-Thierry (DT) features. Finally, we discuss policy formulations in Tetris AI, contrasting deterministic greedy approaches with stochastic methods such as Proximal Policy Optimization (PPO) and REINFORCE, setting the stage for our proposed enhancements in environment design and training efficiency.

The Tetris Game

The Tetris game, designed by Alexey Pajitnov and first released in 1984, involves manipulating seven distinct pieces (I, O, T, L, J, S, Z) through translation and rotation to form complete horizontal lines at the bottom of a grid (typically 10×20 , Fig 1). Completed lines are cleared, and blocks above descend, while uncleared lines accumulate until they exceed the grid’s upper boundary, resulting in a game over. The core objective is to maximize the number of cleared lines.

**Figure 1.** Tetris Game Board with All Pieces

A critical concept in Tetris is the *afterstate*. Given a state s and an action a , the state where the next random piece has not yet been generated after executing action a is called an afterstate. The afterstate reflects the direct result of the current action. As shown in Fig 2, an afterstate (i.e., Fig 2c) is added before the transition from state s to s' . The afterstate combined with the next randomly generated piece constitutes the next state s' . This afterstate directly determines the available space for subsequent pieces, making it a key factor in long-term strategy. This creates a perfect scenario for studying RL applications in complex decision-making problems. By designing efficient reward functions and state representations, researchers can explore optimization methods for RL algorithms in high-dimensional state spaces.

Tetris AI

MDP Formalization The game of Tetris can be described using the MDP framework to model the interaction between

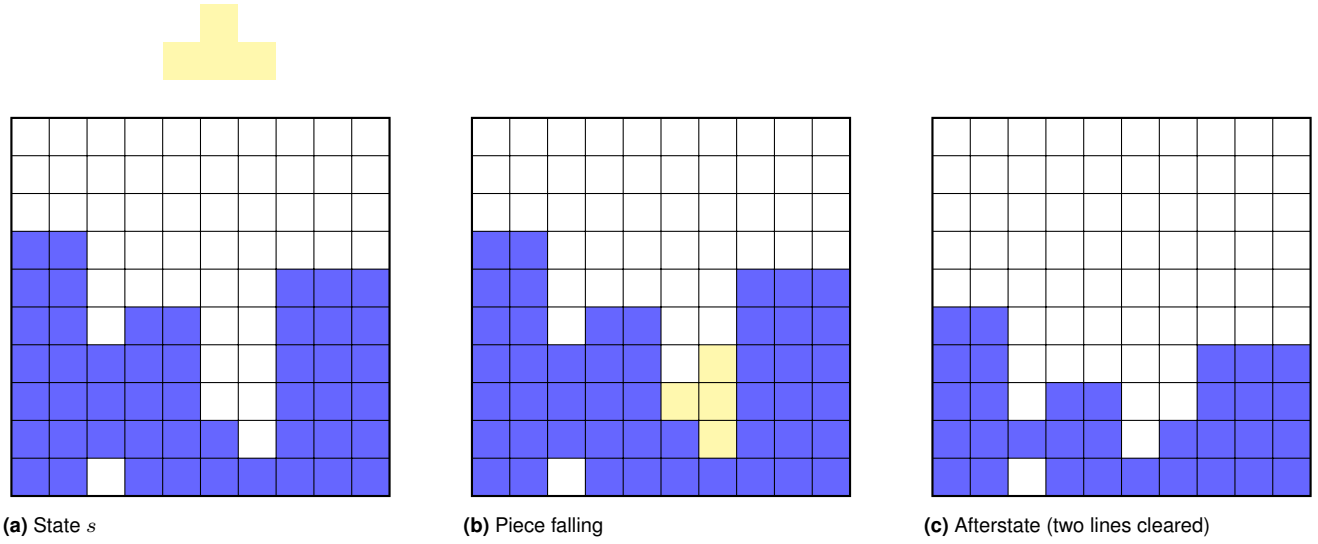


Figure 2. Illustration of the Afterstate in Tetris Game

the agent and the game environment. MDP is a fundamental framework in reinforcement learning used to describe the interaction between an agent and its environment. In the game of Tetris, the MDP can be formally represented by a four-tuple (S, A, R, P) , which includes the following key elements:

- (i) **State Space (S):** Represents the current game situation, including the grid configuration of placed blocks, the type, rotation, and position of the current falling piece, and the next piece in the preview. This defines the complete information set upon which the agent bases its decisions.
- (ii) **Action Space (A):** Encompasses all valid operations on the current piece, defined as combinations of rotation states (0 to 3 rotations) and horizontal positions (within the grid boundaries). Each action (*rotation, translation*) corresponds to a unique placement for the piece.
- (iii) **State Transition Probability (P):** Describes the dynamics of the environment. It is a function $P : S \times A \times S \rightarrow [0, 1]$. $P(s'|s, a)$ denotes the probability of transitioning to state s' after taking action a in state s . In Tetris, transitions are partially deterministic (grid update based on action a) and partially stochastic (the next piece is generated randomly with equal probability from the 7 types).
- (iv) **Reward Function (R):** Quantifies the immediate feedback for the agent. It is a function $R : S \times A \times S \rightarrow \mathbb{R}$. $R(s, a, s')$ represents the scalar reward received after taking action a in state s , resulting in state s' . Its value is typically determined by the number of lines cleared ($l \in \{0, 1, 2, 3, 4\}$) as a direct result of the action.

This MDP formulation enables RL agents to learn optimal placement strategies by maximizing cumulative long-term rewards. In this MDP framework, the agent’s goal is to learn an optimal policy $\pi(a|s)$ —a mapping from states to

actions—that maximizes the expected cumulative reward. RL algorithms for policy learning are broadly divided into *value-based* (e.g., Q-Learning, which learns action values to derive policies indirectly) and *policy-based* (e.g., PPO, which optimizes the policy directly via gradient ascent), with the latter being more suitable for Tetris’s discrete action space and long-episode training.

State and Action Representation Raw state representations (e.g., direct grid arrays) are impractical for Tetris due to their high dimensionality ($10 \times 20 = 200$ binary features). Instead, *feature engineering* simplifies the state by extracting meaningful abstract features. Among the most effective is the DT feature set, which captures critical aspects of board structure and placement quality.

The DT features, building on Dellacherie-Thiery’s 6 original features with 3 extensions^{14,15}, are defined for a grid of width w and height h (cells $b(x, y) \in \{0, 1\}$, where 1 denotes a filled cell) as follows¹⁶:

- (i) **Landing height:** $t_y + (t_h - 1)/2$ (vertical distance from the piece’s centroid to the grid bottom, where t_y is the lowest row and t_h is the piece height).
- (ii) **Eroded piece cells:** $n_r \times n_c$ (contribution to line clears, with n_r = cleared lines and n_c = piece cells in cleared lines).
- (iii) **Row transitions:** Count of filled/empty transitions in rows (including edges treated as filled, computed via XOR).
- (iv) **Column transitions:** Vertical counterpart to row transitions, with similar edge handling.
- (v) **Holes:** Empty cells beneath filled cells in columns, determined using column height $ch(x)$ (maximum filled row in column x).
- (vi) **Board well:** Sum of depths of vertical wells (columns with filled neighbors, boundaries treated as filled).
- (vii) **Hole depth:** Total distance from holes to the nearest filled cell above.

- (viii) **Rows with holes:** Count of rows containing at least one hole.
- (ix) **Pattern diversity:** Number of distinct height differences (-2 to $+2$) between adjacent columns.

These features are combined linearly to estimate state value: $V(s) = \sum_{i=1}^9 \Phi_i \theta_i$, where Φ_i are the DT features and θ_i are learned weights. Two representative weight sets (DT-10 and DT-20) are shown in Table 2.

Table 2. DT-10 and DT-20 weights^{6,7}.

Feature	DT-10 weight	DT-20 weight
Landing height	-2.18	-2.68
Eroded piece cells	2.42	1.38
Row transitions	-2.17	-2.41
Column transitions	-3.31	-6.32
Holes	0.95	2.03
Board well	-2.22	-2.71
Hole depth	-0.81	-0.43
Rows with holes	-9.65	-9.48
Diversity	1.27	0.89

For action representation, each piece’s valid placements (rotation + horizontal position) are mapped to discrete indices. For example, if the maximum number of valid actions across all pieces is $N_{\max}$, actions are encoded as $A = \{0, 1, \dots, N_{\max} - 1\}$.

Policy in Tetris AI The policy function $\pi(a|s)$ defines the probability of selecting action a in state s . Policies in Tetris AI can be broadly categorized into deterministic and stochastic types, each tailored to the game’s discrete action space and afterstate properties. Common in Tetris RL, deterministic greedy policies select the action that maximizes expected returns:

$$\pi(a|s) = \arg \max_{a \in A} Q(s, a) \quad (1)$$

where $Q(s, a)$ is the action-value function, representing the expected cumulative reward of taking action a in state s and following π thereafter. For Dellacherie-Thierry (DT) features, $Q(s, a)$ can be approximated using the state value $V(s')$ of the resulting post-state s' :

$$Q(s, a) \approx R(s, a, s') + \gamma V(s') \quad (2)$$

where $R(s, a, s')$ is the immediate reward (lines cleared), $\gamma \in [0, 1)$ is the discount factor, and $V(s') = \sum_{i=1}^9 \Phi_i(s') \theta_i$. For stochastic policies, adopted in policy-based algorithms like Proximal Policy Optimization (PPO), the policy is parameterized by a set of parameters θ (e.g., weights of a neural network) and outputs the probability of selecting action a in state s :

$$\pi_{\theta}(a|s) = P(a|s; \theta) \quad (3)$$

In the context of Tetris, typical actor network designs for PPO leverage value function evaluations. The standard approach employs an action value-based actor, where the network evaluates the action-value function for each

Algorithm 2.1 REINFORCE Algorithm

```

1: Initialize Actor and Critic networks
2: for each episode do
3:   Compute  $\tau$ ,  $done = false$ ,  $s = env.reset()$ 
4:   while not  $done$  do
5:     Compute  $f(s, a)_{all}$  and  $mask$  based on state  $s$ 
6:     Actor selects action  $a$  using  $f(s, a)_{all}$  and  $mask$ 
7:     Execute  $env.step(a)$ , receive next state  $s'$  and
       reward  $r$ 
8:   end while
9:   Compute return:  $G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ 
10:  Update Actor:  $\theta \leftarrow \theta + \alpha \sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ 
11: end for

```

available action, yielding a Boltzmann-style policy:

$$\pi(a|s) = \frac{\exp(Q(s, a))}{\sum_b \exp(Q(s, b))} \quad (4)$$

This design requires action-dependent inputs (e.g., one-hot encodings of the current tetromino type and action index) in addition to state features. However, a more efficient alternative leverages the afterstate property of Tetris. Since the afterstate abstracts the board configuration post-action but pre-next-piece, it simplifies computation by inherently accounting for expectations over future states: The state-value function of the afterstate represents the expectation over possible next states:

$$V(as) = \sum_{s' \in S} V(s') P(s'|as) \quad (5)$$

This leads to the afterstate-based actor, where actions are selected according to:

$$\pi(a|s) = \frac{\exp(V(f(s, a)))}{\sum_b \exp(V(f(s, b)))} \quad (6)$$

Here, $f(s, a)$ represents the feature representation of the afterstate reached by executing action a in state s . This approach maintains a simpler network architecture while incorporating uncertainty over future pieces. PPO¹⁷ is a policy gradient method that updates policy network parameters by maximizing the expected cumulative reward. Its core innovation is the use of a clipped surrogate objective to constrain update magnitudes, ensuring similarity between new and old policies for enhanced training stability and efficiency. This addresses high variance in policy gradients, a common issue in methods like REINFORCE (detailed below). Compared to Trust Region Policy Optimization (TRPO)¹⁸, PPO reduces computational complexity by approximating the KL-divergence constraint with clipping, making it more scalable for games like Tetris. In this work, we adopt a trajectory-based PPO algorithm as the baseline to train the Tetris agent: for each update, we sample a complete game trajectory to refine the policy network. The specific performance of this trajectory-based PPO baseline (**Algorithm 2.2**) will be elaborated on in the Experiments section. For comparison, we also consider REINFORCE(**Algorithm 2.1**) as a simpler policy gradient baseline. REINFORCE updates the policy using Monte

Carlo estimates of returns, but suffers from high variance without variance reduction techniques like baselines or advantages:

Algorithm 2.2 Trajectory-based PPO Algorithm

```

1: Initialize Actor and Critic networks
2: for each episode do
3:   Set  $t = 0$ ,  $done = false$ ,  $s = env.reset()$ 
4:   while not  $done$  do
5:     Compute  $f(s, a)_{all}$  and  $mask$  based on state  $s$ 
6:     Actor selects action  $a$  using  $f(s, a)_{all}$  and  $mask$ 
7:     Execute  $env.step(a)$ , receive next state  $s'$  and reward  $r$ 
8:   end while
9:   Compute TD error:
10:   $\delta_t = r + \gamma \cdot Critic(f(s_t, a_t)) - Critic(f(s_t))$ 
11:  Compute Generalized Advantage Estimation (GAE):
12:   $A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ 
13:  for each training epoch  $k$  do
14:    Update Actor:
15:     $\theta \leftarrow \theta + \alpha_{\theta} \nabla_{\theta} L_a$ 
16:    where  $L_a = \min \left[ \frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A_t, \right.$ 
17:       $\left. \text{clip} \left( \frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A_t \right]$ 
18:    Update Critic:
19:     $w \leftarrow w + \alpha_w \sum_t \delta_t \nabla_w V_w(s_t)$ 
20:  end for
21: end for

```

Code 1. Piece Definition Class

```

1 public static class Piece {
2     public int[] piece;
3     public int width;
4     public int height;
5
6     public Piece(int[] p, int w, int
7         h) {
8         piece = p;
9         width = w;
10        height = h;
11    }
12
13    public static class Sq {
14        public Piece[] pieces;
15        public int rotationNum;
16        public int actionSize;
17        public int maxHeight;
18
19        public Sq(Piece[] p, int s, int
20            size, int height) {
21            pieces = p;
22            rotationNum = s;
23            actionSize = size;
24            maxHeight = height;
25        }
26    }
27 }

```

Bitboard of the Tetris Game

In this section, we first present the design of Tetris based on bitboard, including the representation of the game board and tetrominoes, as well as the acceleration of various game operations through bitwise operations. Subsequently, we introduce the formal definition of DT features and provide their implementation using bitwise operations. Considering language execution efficiency and type constraints, Java was ultimately selected to implement the bitboard-based Tetris, while also providing a Python interface compliant with the OpenAI Gym standard¹¹.

Bitboard Representation for the Tetris Game

Inspired by the Bitboard concept, the Tetris game board can be redesigned. A 32-bit integer is used to represent the state of a single column on the board. For a column, if there is a block at a certain position from bottom to top, the corresponding bit in the integer from the least significant bit to the most significant bit is set to 1. As shown in Figure 1, in the second column from the left, the first and third rows from the bottom contain blocks. Therefore, the second column can be represented in binary as 0B101, which is 5 in decimal, indicating that there are blocks in the first and third rows of that column. Consequently, a single column of the board can be represented by a 32-bit integer, and the entire board can be represented by a 10-dimensional array of int-type integers. This design approach is applicable to both 10×10 and 10×20 game boards.

There are seven types of Tetris pieces: O, I, S, Z, L, J, and T. Following the design approach for the Tetris game board, the pieces also need to be redesigned based on Bitboard. The definition of the Piece class is shown in Code 1.

The Piece class defines a particular shape of a piece after rotation. The Piece class has three properties: an int

array representing the piece, width indicating the width of the piece in that shape, and height indicating the height of the piece in that shape. Since a piece can have different shapes under different rotations, the Sq class is used for encapsulation, packaging all different shapes of the same piece into a single Sq object. The Piece array is used to store the different shapes of the piece after rotation, with the array length determined by the number of shapes the piece can take. rotationNum represents the number of rotations the piece can undergo, actionSize represents the maximum number of actions available for the piece, and maxHeight represents the maximum height of the piece in different shapes. In addition to the basic form of the piece, the width and height of the piece in that shape are also stored for convenience in subsequent DT feature calculations. Take the O piece as an example. As shown in Figure 3, the first and second columns are both 0B11, so the O piece can be represented as 0B11, 0B11, i.e., 3, 3, and stored in an int-type array.



Figure 3. The O-Piece and Its Rotation(s)

The code implementation for the O-piece is shown in Code 2. It is evident that regardless of rotation, the O-piece has only one shape, i.e., 3, 3. The height and width for this shape are both 2, so only one piece needs to be defined. Since the O-piece has a fixed shape under different rotations, when considering the maximum number of available actions, only horizontal movement needs to be taken into account. On a

10-column grid, it can have up to nine different placement positions.

Code 2. Bitboard Representation of O-Piece

```
1 int[] O_piece1 = {3, 3}; // O-piece
2 Piece[] piece1 = new Piece[1];
3 piece1[0] = new Piece(O_piece1, 2, 2);
4 squares[0] = new Sq(piece1, 1, 9, 2);
```

Unlike the O-piece, the I-piece has two shapes: horizontal and vertical, as shown in Figure 4 for their Bitboard representations.

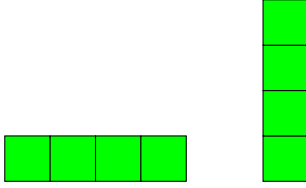


Figure 4. The I-Piece and Its Rotations

The code implementation of the I-piece is shown in Listing 3. `I_piece1` represents the vertical orientation of the I-piece, which can be denoted by $\{0B1111\}$, or equivalently $\{15\}$. `I_piece2` represents the horizontal orientation of the I-piece, which can be denoted by $\{0B1, 0B1, 0B1, 0B1\}$, or equivalently $\{1, 1, 1, 1\}$. In this case, the length of the Piece class array is 2, storing the bitboard representations and the corresponding heights and widths for both the horizontal and vertical orientations. The maximum number of selectable actions is 17 in total, comprising 7 actions in the horizontal orientation and 10 actions in the vertical orientation.

Code 3. Bitboard Representation of I-Piece

```
1 int[] I_piece1 = {15}; // I-piece
2 int[] I_piece2 = {1, 1, 1, 1};
3 Piece[] piece2 = new Piece[2];
4 piece2[0] = new Piece(I_piece1, 1, 4);
5 piece2[1] = new Piece(I_piece2, 4, 1);
6 squares[1] = new Sq(piece2, 2, 17, 4);
```

The S-piece has two shapes, as shown in Figure 5 for their Bitboard representations.



Figure 5. The S-Piece and Its Rotations

The code implementation of the S-piece is shown in Listing 4. `S_piece1` represents the vertical orientation on the left in Figure 5, which can be denoted by $\{0B110, 0B011\}$, or equivalently $\{6, 3\}$. `S_piece2` represents the horizontal orientation on the right in Figure 5, which can be denoted by $\{0B01, 0B11, 0B10\}$, or equivalently $\{1, 3, 2\}$. In this case, the length of the Piece class array is 2. The maximum number of selectable actions is 17 in total, comprising 9 actions

in the vertical orientation and 8 actions in the horizontal orientation.

Code 4. Bitboard Representation of S-Piece

```
1 int[] S_piece1 = {6, 3}; // S-piece
2 int[] S_piece2 = {1, 3, 2};
3 Piece[] piece3 = new Piece[2];
4 piece3[0] = new Piece(S_piece1, 2, 3);
5 piece3[1] = new Piece(S_piece2, 3, 2);
6 squares[2] = new Sq(piece3, 2, 17, 3);
```

The Z-piece has two shapes, as shown in Figure 6 for their Bitboard representations.

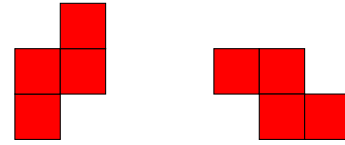


Figure 6. The Z-Piece and Its Rotations

The code implementation for the Z-piece is shown in Code 5. `Z_piece1` corresponds to the vertical placement of the Z-piece shown on the left in Figure 6 and can be represented as $\{0B011, 0B110\}$, i.e., $\{3, 6\}$. `Z_piece2` corresponds to the horizontal placement of the Z-piece shown on the right in Figure 6 and can be represented as $\{0B10, 0B11, 0B01\}$, i.e., $\{2, 3, 1\}$. At this point, the Piece class array has a length of 2. The maximum number of available actions is 17, consisting of nine for the vertical placement and eight for the horizontal placement.

Code 5. Bitboard Representation of Z-Piece

```
1 int[] Z_piece1 = {3, 6}; // Z-piece
2 int[] Z_piece2 = {2, 3, 1};
3 Piece[] piece4 = new Piece[2];
4 piece4[0] = new Piece(Z_piece1, 2, 3);
5 piece4[1] = new Piece(Z_piece2, 3, 2);
6 squares[3] = new Sq(piece4, 2, 17, 3);
```

The L-piece has four shapes, as shown in Figure 7 for their Bitboard representations.

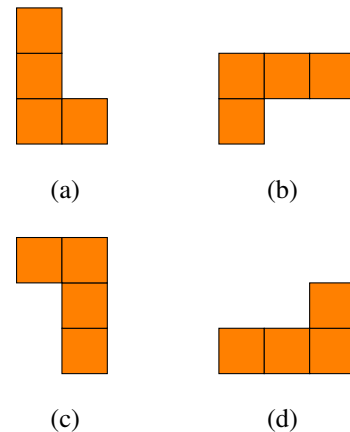


Figure 7. The L-Piece and Its Rotations

The code implementation for the L-piece is shown in Code 6. L_piece1 corresponds to Figure 7(a) and can be represented as {0B111,0B001}, i.e., {7,1}, with nine available actions. L_piece2 corresponds to Figure 7(b) and can be represented as {0B11,0B10,0B10}, i.e., {3,2,2}, with eight available actions. L_piece3 corresponds to Figure 7(c) and can be represented as {0B100,0B111}, i.e., {4,7}, with nine available actions. L_piece4 corresponds to Figure 7(d) and can be represented as {0B01,0B01,0B11}, i.e., {1,1,3}, with eight available actions. The maximum number of available actions is 34.

Code 6. Bitboard Representation of L-Piece

```
1 int[] L_piece1 = {7, 1}; // L-piece
2 int[] L_piece2 = {3, 2, 2};
3 int[] L_piece3 = {4, 7};
4 int[] L_piece4 = {1, 1, 3};
5 Piece[] piece5 = new Piece[4];
6 piece5[0] = new Piece(L_piece1, 2, 3);
7 piece5[1] = new Piece(L_piece2, 3, 2);
8 piece5[2] = new Piece(L_piece3, 2, 3);
9 piece5[3] = new Piece(L_piece4, 3, 2);
10 squares[4] = new Sq(piece5, 4, 34, 3);
```

The J-piece has four shapes, as shown in Figure 8 for their Bitboard representations.

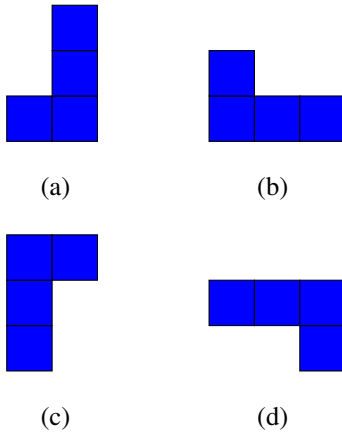


Figure 8. The J-Piece and Its Rotations

The code implementation for the J-piece is shown in Code 7. J_piece1 corresponds to Figure 8(a) and can be represented as {0B001,0B111}, i.e., {1,7}, with nine available actions. J_piece2 corresponds to Figure 8(b) and can be represented as {0B11,0B01,0B01}, i.e., {3,1,1}, with eight available actions. J_piece3 corresponds to Figure 8(c) and can be represented as {0B111,0B100}, i.e., {7,4}, with nine available actions. J_piece4 corresponds to Figure 8(d) and can be represented as {0B01,0B01,0B11}, i.e., {2,2,3}, with eight available actions. The maximum number of available actions is 34.

The T-piece has four shapes, as shown in Figure 9 for their Bitboard representations.

The code implementation for the T-piece is shown in Code 8. T_piece1 corresponds to Figure 9(a) and can be represented as {0B01,0B11,0B01}, i.e., {1,3,1}, with eight available actions. T_piece2 corresponds to Figure 9(b) and can be represented as {0B111,0B010}, i.e., {7,2}, with nine

Code 7. Bitboard Representation of J-Piece

```
1 int[] J_piece1 = {1, 7}; // J-piece
2 int[] J_piece2 = {3, 1, 1};
3 int[] J_piece3 = {7, 4};
4 int[] J_piece4 = {2, 2, 3};
5 Piece[] piece6 = new Piece[4];
6 piece6[0] = new Piece(J_piece1, 2, 3);
7 piece6[1] = new Piece(J_piece2, 3, 2);
8 piece6[2] = new Piece(J_piece3, 2, 3);
9 piece6[3] = new Piece(J_piece4, 3, 2);
10 squares[5] = new Sq(piece6, 4, 34, 3);
```

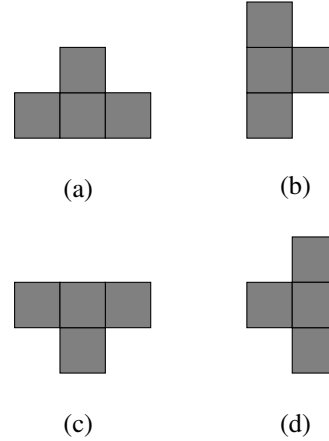


Figure 9. The T-Piece and Its Rotations

available actions. T_piece3 corresponds to Figure 9(c) and can be represented as {0B10,0B11,0B10}, i.e., {2,3,2}, with eight available actions. T_piece4 corresponds to Figure 9(d) and can be represented as {0B010,0B111}, i.e., {2,7}, with nine available actions. The maximum number of available actions is 34.

Code 8. Bitboard Representation of T-Piece

```
1 int[] T_piece1 = {1, 3, 1}; // T-piece
2 int[] T_piece2 = {7, 2};
3 int[] T_piece3 = {2, 3, 2};
4 int[] T_piece4 = {2, 7};
5 Piece[] piece7 = new Piece[4];
6 piece7[0] = new Piece(T_piece1, 3, 2);
7 piece7[1] = new Piece(T_piece2, 2, 3);
8 piece7[2] = new Piece(T_piece3, 3, 2);
9 piece7[3] = new Piece(T_piece4, 2, 3);
10 squares[6] = new Sq(piece7, 4, 34, 3);
```

So far, this paper has presented the Bitboard-based design for the seven types of pieces in the Tetris game. Building upon this representation, the next critical step is to efficiently simulate piece movements and landing detection through bitwise operations.

Piece Landing During the Tetris game process, one of the seven types of pieces is randomly generated. When a piece lands, it must make contact with the existing pieces on the board but not overlap with them. A common approach is to simulate the piece starting from the highest column height within the falling range and descending one step at a time. After each descent, it checks whether any cell overlaps with

the existing pieces on the board. If an overlap occurs, the previous position is the actual landing spot of the piece. From the above process, it is evident that the highest column height within the falling range needs to be quickly identified. Below is an introduction to the algorithm for calculating the column height of a single column. In practice, iterating through all columns within the piece's falling range will yield the highest column height within that range. The pseudocode is shown in **Algorithm 3.1**, with the input being a column's data of type int (a 4-byte integer with 32 bits), and the output being the column height minus 1.

Algorithm 3.1 Binary Search for Column Height

Input: Column data d of the int type

```

1: Initialize  $h = 0$ 
2: if ( $d \& 0xffff0000 \neq 0$ ) then
3:    $d \ggg 16$ 
4:    $h += 16$ 
5: end if
6: if ( $d \& 0xff00 \neq 0$ ) then
7:    $d \ggg 8$ 
8:    $h += 8$ 
9: end if
10: if ( $d \& 0xf0 \neq 0$ ) then
11:    $d \ggg 4$ 
12:    $h += 4$ 
13: end if
14: if ( $d \& 0x0c \neq 0$ ) then
15:    $d \ggg 2$ 
16:    $h += 2$ 
17: end if
18: if ( $d \& 0x02 \neq 0$ ) then
19:    $h += 1$ 
20: end if

```

Output: h

This algorithm outputs the column height minus 1 rather than the original height value to optimize the collision detection process. As shown in Figure 10, if the L-piece is placed in columns one and two in the Bitboard form 4, 7, with column one having a height of 2 and column two a height of 0. Shifting each column's value of the piece left by 2 (equivalent to moving the piece up two positions from the bottom of the board) results in 16, 28, as shown in Figure 10a. Clearly, no collision detection is needed in this case. As shown in Figure 10b, collision detection should actually begin by moving the piece up by column height minus 1 positions, i.e., the bottommost cell of the falling piece should be less than or equal to the highest column.

Another issue to address is how to quickly perform collision detection. Thanks to the Bitboard design of the game pieces and board, collision detection can be achieved by performing a bitwise AND (&) operation between the column value of the board and the column value of the piece. If the result is not zero, it indicates a collision. Taking Figure 10b as an example, the board's first two columns are in state $\{0B00001, 0B00011\}$, and the piece's state is $\{0B01000, 0B01110\}$. After performing the bitwise AND operation, the result is $\{0B00000, 0B00010\}$. Since the result of the bitwise AND operation in the second column

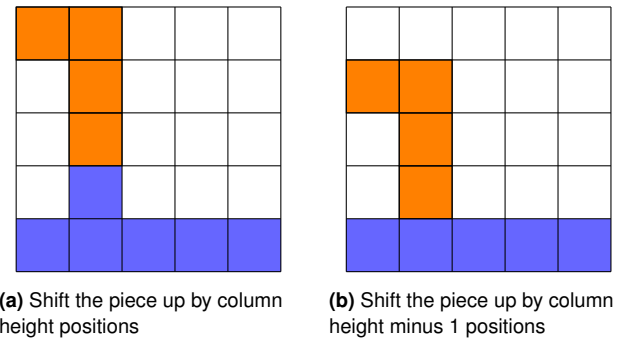


Figure 10. Collision Detection

is not zero, it can be concluded that an overlap has occurred between the piece and the board.

Piece Elimination After a piece lands, it is necessary to check for line clearing operations. The detection process is relatively simple: a variable `deleteLine` initialized to 0xfffff (i.e., 20 ones in decimal, with the number of ones equal to the column height) is used. By performing a bitwise AND operation between the `deleteLine` variable and each column of the board, the rows that need to be cleared are identified as the positions where the bits are 1 in the final result. For a board with a column height of 20, the pseudocode for line clearing is shown in **Algorithm 3.2**, where the int-type state array stores the states of the ten columns. Through `belowTemp`, the blocks below the line to be cleared are preserved in `below`, and through `aboveTemp`, the blocks above the line to be cleared are preserved in `above`. By performing a bitwise AND operation between `below` and (`above >> 1`), the column after clearing the line is obtained. This process is repeated until no more lines need to be cleared.

Algorithm 3.2 Line Clearing

Input: int[] state

```

1: Initialize temp = state[0]
2: while temp  $\neq 0$  do
3:   for each s in state do
4:     temp = temp & s
5:   end for
6:   if temp  $\neq 0$  then
7:     Starting from the highest bit with a value of 1 in
temp, assign all the lower bits to 1
8:     belowTemp = temp >> 1, aboveTemp = ~temp
9:     for each s in state do
10:      below = s & belowTemp, above = s &
aboveTemp
11:      s = below — (above >> 1)
12:    end for
13:  end if
14: end while

```

This section also involves an operation that can be accelerated: how to set all bits from the highest set bit to the end to 1. For reference, see how a 32-bit integer is set from the highest bit to all subsequent bits to 1 in Java's `HashMap`. The pseudocode is shown in **Algorithm 3.3**, with the input being an int-type integer that needs to set all bits from the highest set bit to 1. Since the board's column height

is represented by a 32-bit int variable in this paper's design, this algorithm is suitable.

Algorithm 3.3 Set All Bits from the Highest Bit to 1

Input: temp

```

1: temp = as[i]
2: temp -= temp >> 1
3: temp -= temp >> 2
4: temp -= temp >> 4
5: temp -= temp >> 8
6: temp -= temp >> 16

```

Output: temp

GameOver Determination The operation to determine whether the game is over is relatively simple. For a board with a column height of 20, after each piece lands, a variable initialized to 0xf00000 is used to perform a bitwise AND operation on each bit (checking for blocks in rows 21 to 25). If the result of iterating through all columns is not zero, it indicates that blocks have exceeded the 20th row, and the game is over. The pseudocode is shown in **Algorithm 3.4**, with the input being an int-type array representing the board state and the output being a boolean variable indicating whether the game is over. This algorithm is suitable for a board with a column height of 20; for boards with different column heights, the temp variable should be correspondingly adjusted.

Algorithm 3.4 Determine Game Over

Input: Input int[] state

```

1: Initialize temp = 0xf00000
2: for x in each value in state do
3:   temp = temp & x
4: end for
5: if temp ≠ 0 then
6:   return True
7: else
8:   return False
9: end if

```

DT Features Based on Bitboard

Building upon the "afterstate" concept introduced earlier (Fig. 2), an efficient method for computing DT features using a bitboard-based state representation is presented here. To facilitate the calculation of the DT features, this paper does not only save the states of the 10 columns when designing the int array *state* to represent the state of the Tetris game board. The int array *state* is defined as 15-dimensional, where dimensions 0 to 9 represent the states of the 10 columns from left to right on the board; the 10th dimension is *reward*, indicating the number of lines cleared when a piece falls; the 11th dimension is *score*, indicating the cumulative number of lines cleared; the 12th dimension is *randomBlock*, a randomly generated integer ranging from 0 to 6; the 13th dimension is *dropHeight*, representing the height at which a piece falls, and during the piece overlap detection, the height at which the first overlap occurs is *dropHeight*; the 14th dimension is *deleteLine*, indicating the lines to be cleared,

calculated as shown in the pseudocode in **Algorithm 3.5**. Initially, *state[12]* is initialized to a random integer between 0 and 6, and the rest are initialized to zero. When a piece falls from state *s* to state *s'*, when calculating the state of the 10 columns in state *s'*, the four values other than the random piece are also calculated and saved in the array, along with the new random piece, denoted as the new array *as*. The *as* array is used in the subsequent calculation of the DT features.

Algorithm 3.5 calculate deleteLine

Input: int[] state

```

1: deleteLine = state[0]
2: for i in {1, 2, ..., 9} do
3:   deleteLine = deleteLine & state[i]
4: end for

```

Output: deleteLine

After introducing the state array, we will now explain how the DT features are specifically calculated.

- (i) **Landing height:** The height at which the piece falls, *dropHeight*, is already saved in the state array, and this value is denoted as t_y . When defining the piece class based on the bitboard design, the height of the piece in this shape has already been stored, so the landing height can be directly calculated. In Figure 2c, the height at which the piece falls is 1, and the height of the piece in this shape is 3. Therefore, $f_{\text{landing height}} = 1 + (3 - 1)/2 = 2$.
- (ii) **Eroded piece cells:** The pseudocode is shown in **Algorithm 3.6**. The inputs are an int array *as* and the current falling piece *b*, where the Piece class is the piece class proposed before. The algorithm uses short-circuit programming: if *as[10]* (i.e., reward) is 0, it is clear that no lines have been cleared, so it directly returns 0. *deleteLine* stores the cleared lines. Since the piece is stored aligned with the first row, *deleteLine* is first shifted downward by *dropHeight* to align with the piece. Then, a bitwise AND operation is performed on each column of the piece, and the number of 1s is counted. In Figure 2c, the number of cleared lines is 2, and the number of cells in the piece that participated in the clearing is 3. Therefore, $f_{\text{eroded piece cells}} = 2 \times 3 = 6$.

Algorithm 3.6 Calculating the Eroded piece cells feature

Input: int[] *as*, Piece *p*

```

1: if as[10] == 0 then
2:   return 0
3: end if
4: deleteLine = as[14] >>> as[13], num = 0
5: for i in {0, ..., piece.width} do
6:   temp = deleteLine & p.piece[i]
7:   while temp ≠ 0 do
8:     temp &= (temp - 1)
9:     ++ num
10:  end while
11: end for

```

Output: $num \times as[10]$

- (iii) **Row transitions:** The state array stores the states of 10 columns. According to the formal definition, it is divided into three parts for calculation: counting the transitions between each column from the first to the ninth column and the next column, the number of empty cells in the first column, and the number of empty cells in the tenth column. The pseudocode is shown in **Algorithm 3.7**. For a column-based Tetris game, 0x3ff should be replaced with 0xffff. In Figure 2c, the number of empty cells in the first column is 5, the number of empty cells in the tenth column is 6, and the number of cell transitions between the first to ninth columns and the next column are {0, 4, 2, 0, 2, 1, 2, 0, 0}, respectively. Therefore, $f_{\text{row transitions}} = 5 + 6 + (4 + 2 + 2 + 1 + 2) = 22$.

Algorithm 3.7 Calculating the Row transitions feature

Input: int[] *as*
1: $num = 0, y = as[0] \oplus 0x3ff$
2: **while** $y \neq 0$ **do**
3: $y = y \& (y - 1)$
4: $++ num$
5: **end while**
6: $y = as[9] \oplus 0x3ff$
7: **while** $y \neq 0$ **do**
8: $y = y \& (y - 1)$
9: $++ num$
10: **end while**
11: **for** i in {0, ..., 8} **do**
12: $y = as[i] \oplus as[i + 1]$
13: **while** $y \neq 0$ **do**
14: $y \& = (y - 1)$
15: $++ num$
16: **end while**
17: **end for**
Output: *num*

- (iv) **Column transitions:** The pseudocode is shown in **Algorithm 3.8**. By shifting the column up one cell using $as[i] \ll 1$, and adding 1 because the bottom of the board is considered to have a block, the state of the first row is set to have a block after the shift. The XOR operation can then be used to count the transitions in each column. In Figure 2c, the number of transitions in each column from the first to the tenth column is {1, 1, 3, 1, 1, 1, 1, 1, 1, 1}, respectively. Therefore, $f_{\text{column transitions}} = 1 + 1 + 3 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 12$.
- (v) **Holes:** The pseudocode is shown in **Algorithm 3.9**. For each column, starting from the highest 1 and filling all positions below it with 1, the number of holes is counted by XORing with the original column state and counting the number of 1s. Clearly, in Figure 2c, the number of holes is 1.
- (vi) **Board well:** The pseudocode is shown in **Algorithm 3.10**. First, a new array *exState* is constructed, where the first and twelfth dimensions are all 1 (if the column height is 20, replace 0x3ff with 0xffff), and the second to eleventh dimensions are the int values representing

Algorithm 3.8 Calculating the Column transitions feature

Input: int[] *as*
1: $num = 0$
2: **for** i in {0, ..., 9} **do**
3: $y = as[i] \oplus ((as[i] \ll 1) + 1)$
4: **while** $y \neq 0$ **do**
5: $y = y \& (y - 1)$
6: $++ num$
7: **end while**
8: **end for**
Output: *num*

Algorithm 3.9 Calculating the Holes feature

Input: int[] *as*
1: $num = 0$
2: **for** i in {0, ..., 9} **do**
3: **if** $as[i] > 1$ **then**
4: $y = as[i]$
5: $y | = y \gg 1$
6: $y | = y \gg 2$
7: $y | = y \gg 4$
8: $y | = y \gg 8$
9: $y | = y \gg 16$
10: $y = y \oplus as[i]$
11: **while** $y \neq 0$ **do**
12: $y = y \& (y - 1)$
13: $++ num$
14: **end while**
15: **end if**
16: **end for**
Output: *num*

the ten-column states. Then, each column is traversed to calculate the cumulative depth of each well. In Figure 2c, the cumulative well depths for each column are {0, 0, 1 + 1, 0, 0, 1, 0, 0, 0, 0}. Therefore, $f_{\text{board wells}} = 2 + 1 = 3$.

- (vii) **Hole depth:** The pseudocode is shown in **Algorithm 3.11**. For each column, starting from the bottom and moving upwards, the first hole is found, and then the number of cells above the hole that contain blocks is counted. In Figure 2c, there is only one hole in the third column with one block above it. Therefore, $f_{\text{hole depth}} = 1$.
- (viii) **Rows with holes:** The pseudocode is shown in **Algorithm 3.12**. Each column is traversed to find rows with empty cells, which are stored in *y2*. Finally, the total number of rows with empty cells in *y2* is counted. In Figure 2c, there is only one row with a hole. Therefore, $f_{\text{rows with holes}} = 1$.
- (ix) **Pattern diversity:** When calculating the height differences between adjacent columns, the height of each column needs to be calculated. This can be done using **Algorithm 3.1**, which is denoted as the *searchHeight* function. The pseudocode for calculating Pattern diversity is shown in **Algorithm 3.13**. Since *searchHeight* returns the column height

Algorithm 3.10 Calculating the Board well feature

Input: int[] *as*

```

1: num = 0, int[] exState = newint[12]
2: exState[0] = exState[11] = 0x3ff
3: for i in {1, ..., 10} do
4:   exState[i] = state[i - 1]
5: end for
6: for i in {1, ..., 10} do
7:   for j in {0, ..., 9} do
8:     if ((exState[i] >> j) & 1) == 0 then
9:       ++ y
10:      if ((exState[i - 1] >> j) & 1) == 1 and
          ((exState[i + 1] >> j) & 1) == 1 then
11:        num = num + y
12:      end if
13:    else
14:      y = 0
15:    end if
16:  end for
17:  y = 0
18: end for

```

Output: num

Algorithm 3.11 Calculating the Hole depth feature

Input: int[] *as*

```

1: num = 0
2: for i in {0, ..., 9} do
3:   y =  $\oplus(as[i] \oplus (as[i] + 1))$ 
4:   y = as[i] & y
5:   while y ≠ 0 do
6:     y = y & (y - 1)
7:     ++ num
8:   end while
9: end for

```

Output: num

Algorithm 3.12 Calculating the Row with holes feature

Input: int[] *as*

```

1: num = 0, y2 = 0
2: for i in {0, ..., 9} do
3:   if as[i] > 1 then
4:     y1 = as[i]
5:     y1 = y1 >> 1
6:     y1 = y1 >> 2
7:     y1 = y1 >> 4
8:     y1 = y1 >> 8
9:     y1 = y1 >> 16
10:    y1 = y1  $\oplus$  as[i]
11:    y2 = y2 | y1
12:  end if
13: end for
14: while y2 ≠ 0 do
15:   y2 = y2 & (y2 - 1)
16:   ++ num
17: end while

```

Output: num

result of *searchHeight*, the column height is set to -1. In Figure 2c, the height differences between each column and the next column are {0, 3, -1, 0, 2, -1, -2, 0, 0}. The unique values are {-2, -1, 0, 2}. Therefore, $f_{\text{pattern diversity}} = 4$.

Algorithm 3.13 Calculating the Pattern diversity feature

Input: int[] *as*

```

1: int[] h = newint[10], int[] count = {0, 0, 0, 0, 0}, int[]
   p = {-2, -1, 0, 1, 2}, int num = 0
2: for i in {0, ..., 9} do
3:   if as[i] ≠ 0 then
4:     h[i] = searchHeight(as[i])
5:   else
6:     h[i] = -1
7:   end if
8: end for
9: for i in {0, ..., 8} do
10:  h1 = h[i] - h[i + 1]
11:  for j in {0, ..., 4} do
12:    if h1 == p[j] then
13:      ++ count[j]
14:    end if
15:  end for
16: end for
17: for i in {0, ..., 4} do
18:   if count[i] ≠ 0 then
19:     ++ num
20:   end if
21: end for

```

Output: num

Python interface

In the field of AI today, deep reinforcement learning (DRL) algorithms based on neural networks have demonstrated significant potential in game development and training. Although the game environment is implemented in Java, Python offers unparalleled advantages for AI training. The rise of open-source frameworks (e.g., PyTorch and TensorFlow) has equipped AI developers with powerful and user-friendly API interfaces. For example, PyTorch allows for easy implementation of the required network structures, and gradient calculations via backpropagation can be performed with just a few API calls. Therefore, the final solution adopted in this paper is as follows: The bitboard-based Tetris game environment is implemented in Java, while each operation is invoked through Python, utilizing the open-source library Jpype¹³ version 1.5.0. Jpype is a tool for invoking Java code within the Python environment, enabling Python programs to directly call Java classes and their methods. JPyype supports data type conversions between Python and Java (e.g., converting a Python list to a Java array) and enables the direct import of trained weights into the Java-implemented bitboard-based Tetris game for testing. This approach leverages the performance advantages of the Java language while taking full advantage of the convenience of Python in AI development. For ease of use, a Tetris class is implemented in Python following the OpenAI Gym standard interface. This class is globally unique and initializes the Java

minus 1, for a column with no blocks, to align with the

Virtual Machine (JVM) and the Java-implemented classes upon initialization. The JDK version used by the JVM is consistent with the version used to compile the Java classes, both using openjdk-21.0.1. The following API interfaces are exposed and provided in Python:

- (a) `Tetris.reset()`: Initializes the Tetris game environment and returns a 15-dimensional integer array `state`. The definition of the `state` array is the same as described in Section before.
- (b) `Tetris.isFinal()`: Takes `state` as input and returns a boolean variable indicating whether the game is over.
- (c) `Tetris.parallel_episode()`: Takes the linear weights of the network as input, tests 10,000 games in parallel in Java, and returns the average score of these games. This method is only applicable to the 9-dimensional weights trained with DT features.
- (d) `Tetris.get_9feature()`: Takes `state` as input and returns the DT features and mask for up to 34 possible successor states based on the given `state`.
- (e) `Tetris.step()`: Takes `state` and an action `a` as input, and returns the next state array, the reward for the action, and a boolean indicating whether the game is over.

Buffer-based PPO for Tetris AI

The previous section implemented the Tetris game using bitboard, aiming to achieve significant acceleration over traditional grid-based storage and resolve its computational bottlenecks. To train high-performance agents on the bitboard-based implementation, we designed a buffer-based PPO algorithm that evaluates afterstates, which can rapidly train a well-performing agent. Given the excessive training time required for standard 10×20 boards and the prevalent use of more challenging 10×10 mini-boards in current research, this section adopts 10×10 boards for experiments to balance training efficiency and research representativeness.

Afterstate-Based Actor Network

In the field of reinforcement learning, the Actor network is a key component in policy gradient-based algorithms, whose main function is to learn and output the probability distribution of taking each action under a given state. In Tetris, environmental randomness primarily stems from the generation of the next block type, whereas the placement of the current block and its impact on the board topology exhibit deterministic transition characteristics after an action is executed. Compared to directly estimating the action-value function $Q(s, a)$, value evaluation based on afterstates can explicitly decouple the deterministic outcome of the agent's decision from the stochastic disturbances of the environment system. From the theoretical perspective of policy gradients, the traditional $Q(s, a)$ implicitly requires taking expectations over all subsequent states corresponding to random blocks during updates, which can introduce substantial estimation variance when sampling is insufficient. Since the mapping

$s \times a \rightarrow as$ is deterministic, the afterstate value function $V(as)$ can directly leverage deterministic board features for updates, with expectation calculations applied only to the long-term stochastic distribution of subsequent events. This decoupling mechanism effectively reduces the search space of the value function, significantly lowers the variance of gradient estimates, and thereby enhances the stability of policy updates. This property is particularly crucial in the Tetris task. The line clears and sudden changes in the board state resulting from block placement exhibit highly regular geometric patterns. The Actor network based on afterstates shifts its learning focus from 'evaluating the quality of action-block combinations' to 'evaluating the quality of board configurations.' This not only simplifies the difficulty of feature mapping but also enables the network to achieve deep modeling of board structures with a smaller parameter scale, significantly improving sample utilization efficiency, which will be comparatively analyzed and validated in the experiments presented in the next section.

To effectively harness the parallel computing capabilities of GPUs for the Actor implementation in Tetris, specific optimizations tailored to the game's characteristics are essential. For instance, the Actor network evaluating afterstates shares similarities with those assessing action-value functions. Typically, an Actor network takes a state s or its features as input and outputs the probability distribution of actions a under s . However, in Tetris, the input is uniquely processed: it comprises afterstate features for all feasible actions a from state s , preprocessed with feature extraction, concatenation, and a mask to enhance action selection based on afterstate data. The key steps are:

- (i) **Feature extraction:** Compute afterstate features for each feasible action (e.g., 9 for the O-piece), with infeasible actions assigned arbitrary values (eliminated by the mask, thus irrelevant to softmax output). This chapter uses DT features to quantify game progress.
- (ii) **Feature concatenation:** Combine afterstate features into a 306-dimensional input vector $f(s, a)_{\text{all}}$, accommodating up to 34 actions, with masking for infeasible actions varying by piece (9 to 34).
- (iii) **Masking mechanism:** Apply a mask (1 for feasible, 0 for infeasible) to set infeasible action probabilities to 0 via negative infinity, ensuring only feasible actions are considered.

The Actor structure for afterstate evaluation is depicted in Fig. 11, where input is a 306-dimensional feature and $n(\geq 1)$ is the batch size. For in-game decisions, only current afterstates are input ($n = 1$); for policy updates, batch processing uses trajectory or buffer data. To boost sampling efficiency and utilize GPU parallelism, CPU handles game sampling, while GPU manages agent updates. Through reshape operations, a shared linear layer evaluates all afterstates, masks out infeasible actions, and applies softmax to yield action probabilities.

Buffer-Based PPO

In reinforcement learning, particularly for long-episode games like Tetris, training efficiency critically depends

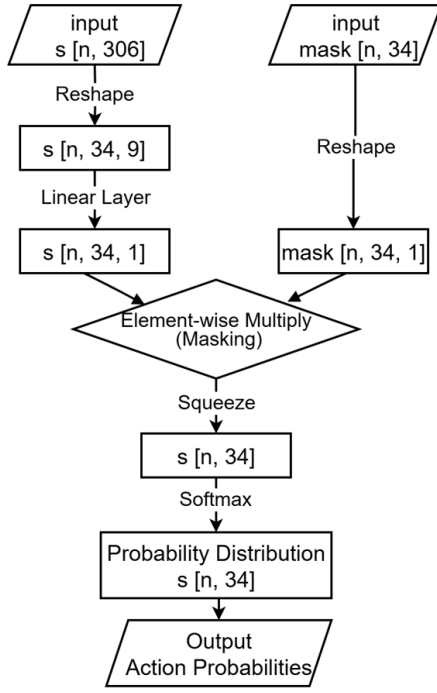


Figure 11. The feedforward process of the Actor evaluating afterstates

on balancing sampling and policy update times. Standard trajectory-based PPO—which collects a full episode before performing updates—suffers from a severe imbalance: sampling dominates computation, while policy updates consume negligible effort. To address this, we propose a Buffer-based PPO algorithm to mitigate the time disparity. The core idea is to adjust the training pace, shifting from the trajectory-based PPO’s approach of waiting for a full game before updating, to initiating training once the Buffer collects a batchSize number of samples. During each training cycle, the process loops epoch times, randomly sampling miniBatchSize data from the Buffer for training. This method aligns the frequency of sampling and policy updates, enhancing the agent’s training speed. The pseudocode for the Buffer-based PPO algorithm is provided in **Algorithm 4.1**.

With the update shifting from episode-based to Buffer-based, the learning rate decay is adjusted to linear decay, as shown in Eq.(7), applied to both Actor and Critic networks. This approach suits Buffer-based training, offering a high initial learning rate to speed up early parameter updates, then gradually reducing it to stabilize updates, prevent oscillations, and enhance convergence to an optimal solution.

$$\alpha = \alpha_{\text{init}} \times \left(1 - \frac{\text{currentStep}}{\text{totalSteps}}\right) \quad (7)$$

Experiments

In this section, we present a series of experiments designed to validate the proposed bitboard-based Tetris implementation and evaluate the performance of various policy optimization approaches. First, we assess the correctness and efficiency of the bitboard implementation by replicating prior benchmarks and comparing execution speeds. Subsequently, we clarified the criteria for selecting

Algorithm 4.1 Buffer-based PPO Algorithm

```

1: Initialize Actor, Critic networks, and Replay Buffer, set step = 0
2: while step ≤ totalStep do
3:   done = false, s = env.reset(), set Buffer.size = 0
4:   while not done do
5:     Compute  $f(s, a)_{\text{all}}$  and  $\text{mask}$  based on state  $s$ 
6:     Actor selects action  $a$  using  $f(s, a)_{\text{all}}$  and  $\text{mask}$ 
7:     Execute  $\text{env.step}(a)$ , receive next state  $s'$  and reward  $r$ 
8:     Store transition data in Buffer for training
9:     if Buffer.size == batchSize then
10:      for each training epoch  $k$  do
11:        Randomly sample a  $\text{miniBatchSize}$  from Buffer
12:        Compute TD error:
13:           $\delta_t = r + \gamma \cdot \text{Critic}(f(s_t, a_t)) - \text{Critic}(f(s_t))$ 
14:        Compute Generalized Advantage Estimation:
15:           $A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ 
16:        Update Actor:
17:           $\theta \leftarrow \theta + \alpha_{\theta} \nabla_{\theta} L_a$ 
18:          where  $L_a = \min \left[ \frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A_t, \right.$ 
19:             $\left. \text{clip} \left( \frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A_t \right]$ 
20:        Update Critic:
21:           $w \leftarrow w + \alpha_w \sum_t \delta_t \nabla_w V_w(s_t)$ 
22:      end for
23:      Reset Buffer.size = 0
24:    end if
25:  end while
26: end while
  
```

evaluation targets and learning algorithms. At the evaluation level, both action-value and afterstate representations were initially identified as core indicators, while at the algorithmic level, basic reinforcement learning and Proximal Policy Optimization (PPO) were compared in parallel. Guided by a greedy selection strategy that prioritizes higher-performing configurations, the experimental results demonstrated that afterstate evaluation provided superior effectiveness over action-value estimation, leading to its adoption as the unified evaluation indicator in subsequent experiments. Similarly, PPO outperformed basic reinforcement learning and was established as the core algorithmic framework. Building upon these selections, further optimization and refinement were pursued. Beginning with conventional trajectory-based PPO, we introduced a buffer mechanism to enhance sample utilization and update efficiency, ultimately forming an improved buffered PPO algorithm that achieved a better balance between learning stability and computational efficiency.

Validation of Bitboard-Based Implementation on Tetris

Benchmark Replication for Correctness Victor Gabillon et al.⁶ achieved an average score of 4200 using the CBMPI algorithm and DT features on a 10×10 board, with the best training run scoring 5200. Gabillon et al. provided two sets of weights, DT-10 and DT-20, in their paper, as shown in Table 2. Gabillon tested each set of weights over 10,000 games. The DT-10 weights achieved an average score of approximately 5000 on the 10×10 board and about 29,000,000 on the 10×20 board. The DT-20 weights achieved an average score of approximately 4300 on the 10×10 board and about 51,000,000 on the 10×20 board.

This paper also tests based on these two sets of weights to verify the correctness of the game environment. On the 10×10 board, 10,000 games were tested for each set of weights, and the results are shown in Table 3. DT-10 and DT-20 directly select the action with the highest linear sum of DT feature values based on all subsequent states' features $f(s, a)$, consistent with the method in the original paper. The test results are consistent with the conclusions given in the original paper.

On the 10×20 board, 10 games were tested for each set of weights, and the results are shown in Table 3, verifying that the Tetris game design based on bitboard in this paper is feasible on both 10×10 and 10×20 boards.

Table 3. Scores with different weights on different board sizes (Mean(SD))

Board Size	Weights	Gabillon et al. (2013) ⁶	Our bitboard impl
10×10 (10,000 games)	DT-10	5,000(-)	5,152.02(5,137.51)
	DT-20	4,300(-)	4,188.61(4,153.42)
10×20 (10 games)	DT-10	29,000,000(-)	54,377,543(54,353,156)
	DT-20	51,000,000(-)	75,093,204(65,000,224)

Runtime Efficiency Comparison A comparison is made between the open-source `gym-tetris` implementation and our bitboard-based Tetris implementation, described in detail below:

- (i) **Standard open-source implementation:** `gym-tetris`¹¹ provides an OpenAI Gym interface based on the `nes-py` emulator for playing Tetris on the Nintendo Entertainment System (NES). This implementation supports reinforcement learning research and multiple game modes with different reward mechanisms. Two main modes are available: A-type (standard endurance Tetris) and B-type (arcade-style, requiring a set number of lines to win). Each mode includes four variants (v0-v3) with different reward structures (e.g., scoring, line-clear bonuses, or penalties for board height). In our experiments, we use the endurance-type Tetris with a line-clear reward function, corresponding to the `TetrisA-v1` version. Although the library supports rendering, image rendering is disabled during training due to high computational cost.
- (ii) **Tetris based on bitboard:** To maximize performance, we implemented Tetris using bitboards. Considering the efficiency limitations of Python for bitwise operations, this implementation was completed in Java, ensuring high execution speed and stability.

For performance testing, each implementation was sampled for 10,000 steps, with the agent selecting actions randomly to avoid time overhead from decision-making. The experiments were conducted on a Windows 11 machine with an AMD R7-7735H CPU and 16GB of RAM, which was used for all subsequent tests. The time comparison of the three implementations is shown in Table 4. It is evident that the bitboard-based Tetris implementation has a significant speed advantage. For 10,000 samples, it runs approximately 53 times faster than the standard OpenAI Gym implementation. Even when called via Python, it achieves around 22 times the speed of the standard

implementation, despite some performance loss due to thread communication.

Table 4. Time comparison of Tetris implementations for 10,000 samples

Implementation	Time for 10,000 samples
OpenAI Gym ¹¹	12.92 seconds
Bitboard Tetris (ours)	0.24 seconds
Bitboard Tetris via Python (ours)	0.59 seconds

Evaluation of PPO Variants for Tetris Agents

Assessment of Afterstate vs. Action-Value To validate the afterstate evaluating Actor, experimental tests are conducted in this section. The REINFORCE algorithm is employed to eliminate the influence of the Critic network. Additionally, to prevent performance discrepancies arising from differences in neural network architectures, only linear layers with dimensionality identical to the input vector are utilized. To accelerate the convergence of the *softmax* function, an annealing factor τ is used in the experiments to speed up convergence. The values input to the *softmax* function are first divided by τ . Here, $\tau(i)$ represents the temperature at the i -th episode, controlling the balance between exploitation and exploration. τ_0 denotes the initial temperature, and τ_k controls the annealing rate.

$$\tau(i) = \frac{\tau_0}{1 + \tau_k \times i} \quad (8)$$

The parameters include a discount factor of $\gamma = 1$ (undiscounted returns), an initial temperature of $\tau_0 = 0.5$, an annealing rate of $\tau_k = 0.00025$, and a learning rate of $\alpha = 1e - 3$. For the comparison, we evaluate two models: the action-value Actor with a 48-dimensional input and the afterstate Actor with a 9-dimensional input, both utilizing linear layers to avoid architecture bias. Due to the long training time of the REINFORCE algorithm, the experiment is conducted over two independent trials. The final results are presented as a mean-variance plot in Figure 12.

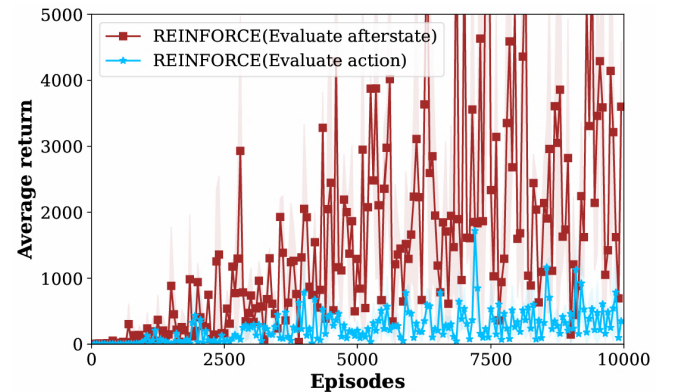


Figure 12. Comparison of Action-Value Evaluating Actor and Afterstate Evaluating Actor

The experimental results indicate that the Actor evaluating afterstates utilizes fewer weights and achieves significantly better performance than the Actor evaluating the action-value function. Analyzing the reasons for this, the placement of a piece can trigger drastic changes in the spatial

structure, creating state transitions far more complex than those assumed by traditional models. Although the DT features numerically represent the current board state, they must contend with dynamic changes (e.g., line-clear cascades and spatial reconfiguration) that occur upon tetromino placement. The traditional method of evaluating action-value, due to its lack of a targeted environmental modeling mechanism, struggles to capture the patterns of state evolution in real-time, leading to delayed policy updates. Although theoretically the expressive power could be enhanced by stacking more network layers, this "brute-force" solution not only significantly increases computational costs but is also highly prone to overfitting, and thus is not a reasonable path for algorithm optimization. Therefore, in subsequent experiments of this paper, the Actor that evaluates afterstates is selected.

Performance of Trajectory-Based PPO Baseline The Trajectory-Based PPO implementation serves as the primary baseline, representing the standard on-policy training paradigm commonly used in reinforcement learning. The experimental parameter settings are shown in Table 5.

Table 5. Trajectory-Based PPO hyperparameters

Param	Value	Param	Value	Param	Value
γ	0.99	τ_0	0.5	α_θ	3e-4
λ	0.99	τ_k	0.00025	α_ω	3e-4
epoch	10	ϵ	0.1	episode	12,500

The experiment was independently repeated 5 times to reduce randomness, and the final results are plotted as the mean-variance graph in Fig. 13:

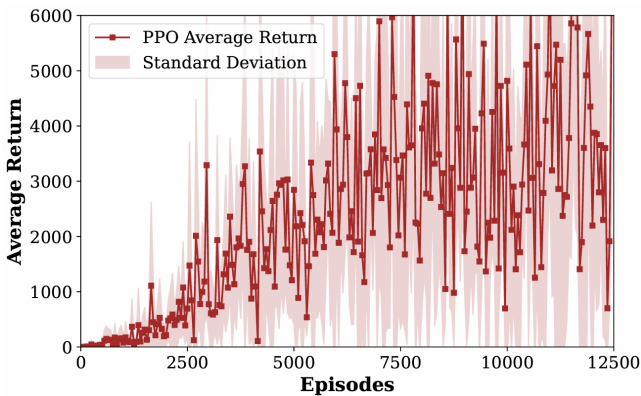


Figure 13. Trajectory-Based PPO Algorithm

As can be seen from the figure, the trajectory-based PPO algorithm achieves significant results when training Tetris game agents. As the number of training rounds increases, the average reward gradually rises. The average value of the rewards from the last 150 episodes of five independent training runs is 3840.30, which is taken as the average convergence value of the training. By combining the PPO algorithm with the Actor that evaluates afterstates, results close to those of CBMPI are achieved.

Efficiency Gains with Buffer-Based PPO The experimental parameter settings are shown in Table 6.

In the previous trajectory-based PPO algorithm, the mean-variance plot is drawn using the return per episode as the

Table 6. Buffer-based PPO hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
γ	0.99	ϵ	0.2	batchSize	2,048
λ	0.99	α_θ	3e-4	miniBatchSize	256
epoch	10	α_ω	3e-4	totalSteps	61,440

evaluation metric during training. For the Buffer-based PPO algorithm, the agent is tested for 50 episodes after each training cycle, with experiments independently repeated 5 times to reduce randomness. The final results are presented as a mean-variance plot in Fig 14.

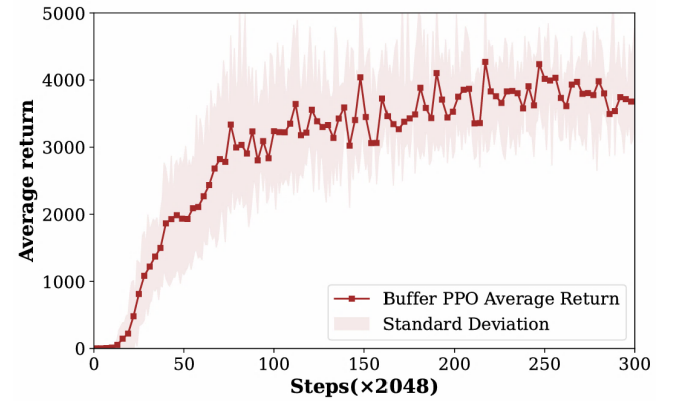


Figure 14. Buffer-Based PPO Algorithm

The average return over the last 20 episodes across five independent runs is 3829.04, used as the mean convergence value. While the Buffer-based PPO does not exceed the trajectory-based PPO in final scores, it markedly improves efficiency. For example, as shown in Table 7, one run took 112 seconds for sampling and 54 seconds for updates, totaling 166 seconds—about 66 times faster than the trajectory-based approach. The update time proportion rose from 3.97% to 32.53%, showing effective balance between sampling and updates, thus boosting training efficiency.

Table 7. Time cost comparison of PPO variants

Algorithm	Sampling	Policy Updates	Total	Update Ratio
Trajectory PPO	10,536s	436s	10,972s	3.97%
Buffer PPO	112s	54s	166s	32.53%

As shown in Table 8, from the perspective of training steps, the trajectory-based PPO algorithm utilized 69,046,726 steps, while the Buffer-based PPO required only 61,440 steps—a difference of approximately 1124 times. Despite this vast gap, the final training performance remains nearly identical. A test of 10,000 episodes was conducted using the better training result from one of the runs, yielding an average score of 4124.47, and the weights of this training result are shown in Table 9.

Table 8. Step cost comparison of PPO variants

Algorithm	Total Steps	Avg. Converged Score
Trajectory PPO	69,046,726	3,840.30
Buffer PPO (optimized)	61,440	3,829.04

This significant difference in training steps can be attributed to two phases:

Table 9. Best weights obtained from PPO training

Feature	Weight
Landing height	-0.51
Eroded piece cells	0.16
Row transitions	-0.40
Column transitions	-0.75
Holes	-0.18
Board well	-0.39
Hole depth	-0.17
Rows with holes	-0.83
Diversity	0.36

- (i) **Early Training with Poor Sample Quality:** In Tetris, each piece is 4 cells, suggesting an average of 2.5 pieces per cleared row, yielding about 1 point per 2.5 steps. Early on, the agent's random action selection results in low scores (often single digits or zero), producing poor-quality samples with little value for policy optimization. The trajectory-based PPO, sampling full games, generates only tens of steps per game with minimal strategic insight, leading to ineffective updates. This necessitates repeated sampling of low-quality data, inflating step counts, as reflected by slow, low step growth in early training.
- (ii) **Late Training with Suboptimal Update Timing:** As training progresses, the agent's improved strategy yields higher scores (e.g., 1250 points over 4000-5000 steps). Valuable samples accumulate mid-game, but the trajectory-based PPO waits for game completion to update, wasting time on potentially suboptimal actions. In contrast, the Buffer-based PPO updates promptly after collecting sufficient samples, leveraging data more efficiently and reducing the step gap in later stages.

Generalization Verification All experimental training in this paper is conducted based on a 10×10 mini-board and a random block generator, without designing an independent training process for the standard 10×20 board. During training on the 10×10 mini-board, the model gradually learns core operational logics including block movement, elimination judgment, and layout optimization. Such logics are partially decoupled from the board size, endowing the trained model with cross-size generalization and transfer capability that can be directly applied to the standard 10×20 board scenario. Table 10 presents the test results on the standard board. Considering performance overhead, the experiment is run independently 100 times and the average value is taken.

Table 10. Scores of two PPO algorithms on the 10×20 game board (Mean(SD))

Algorithm	Score
Trajectory PPO	32,110,545.18(30,425,374.19)
Buffer PPO	13,936,917.72(13,198,381.24)

The results show that although the model can run stably and yield valid scores on the standard board, after direct cross-size transfer, its performance is significantly inferior

to algorithms such as CBMPI compared with the training effect on the 10×10 board. This phenomenon mainly stems from two factors: first, the state space and the number of interactive steps per game of the 10×20 board increase significantly, imposing higher requirements on the model's long-term planning ability and value estimation accuracy; second, the optimal layout strategy (e.g., compact stacking) for the 10×10 board cannot fully adapt to the vertical spatial characteristics of the standard board, limiting strategy applicability. Meanwhile, when extending the bitboard technique and Buffer mechanism proposed in this paper to the standard 10×20 board, two core challenges must be addressed: the significant increase in state space and interactive steps, which may greatly raise the average number of interactive steps per game, leading to distribution shift of Buffer samples and further affecting the stability of policy update; and value estimation bias caused by long-sequence decision-making. The longer game cycle of the standard board amplifies cumulative errors of long-term rewards, demanding higher accuracy in post-state value evaluation.

Moreover, to verify the adaptability of the proposed algorithm to different block generation rules, this experiment adopts three typical rules: Random, 7-Bag, and adversarial Z/S block sequences. The average score of the agent under each rule serves as the core evaluation metric. Random is used as the performance baseline, 7-Bag is the uniform random block generation rule commonly used in engineering, and the adversarial Z/S sequence is a deliberately designed harsh block generation rule to test the agent's anti-interference ability and robustness. In the experiment, 10,000 independent test rounds are performed on the 10×10 board for each rule, and the average of multiple test results is taken as the final data to eliminate the impact of random errors on experimental results. Comparative experiments are completed by three agents under the same experimental environment, parameter configuration, and test rounds, with the results shown in Table 11.

Table 11. Scores of agents under different block generation rules (Mean(SD))

Block Generation Rule	CBMPI	Trajectory PPO	Buffer PPO
Random (Baseline)	4,300	4,965.44 (4884.14)	3,590.90 (3545.05)
7-Bag	251,501.20	173,489.37 (177,212.68)	198,206.23 (198,903.07)
Adversarial Sequence	75	34(0)	53(0)

The results indicate that the performance of the three algorithms is highly consistent with changes in block generation rules: all agents achieve optimal performance under the widely used 7-Bag uniform rule in engineering; under the adversarial sequence rule, all agents drop to extremely low performance levels, showing sensitivity to harsh block sequences. All agents exhibit obvious performance shortcomings under the adversarial sequence rule, and robustness optimization research can be carried out in this direction in future work.

Conclusion

This paper proposes an efficient bitboard-based implementation scheme for Tetris, which significantly enhances the

training capability of reinforcement learning agents for complex games. By redesigning the board and block structures with bitboards and leveraging bitwise operations to accelerate core processes (including block falling, collision detection, line clearing, and DT feature extraction), its performance is improved by 53 times compared to OpenAI Tetris-Gym. With the performance advantages of bitboards and Python-Java integration implemented via Jpytype, rapid prototype development and experimental verification can be achieved. On this basis, this paper designs an afterstate-based evaluation Actor network and a buffer-optimized PPO algorithm, effectively addressing the unique challenges posed by the massive state space and long-term planning in Tetris.

It should be noted that the method proposed in this paper does not take pursuing the absolute highest score as the sole objective, but focuses on achieving competitive policy performance at extremely low time and computing power costs. Experimental results show that the buffer-based PPO algorithm achieves an average score of 3829 on the 10×10 board with only 61,440 interaction steps and approximately 3 minutes of training time. Although its absolute highest score is slightly lower than some state-of-the-art (SOTA) methods that rely on hundreds of millions of samples for training, its time cost advantage holds significant engineering application value.

Through the combination of underlying bitboard acceleration and high-level buffer mechanisms, the framework in this paper enables a complete training process to be completed within a minute-level time scale, thereby significantly improving the practical value of Tetris as a rapid verification platform for reinforcement learning. Future research work can focus on the following directions to provide references for subsequent algorithm optimization: First, explore feature fusion strategies by combining traditional DT features with deep learning features, compare the performance differences of different feature combinations in state representation accuracy and training efficiency, and further enhance the model's ability to capture complex board states; Second, optimize the network structure design by attempting to introduce deep network structures such as MLP and Transformer, or integrate attention mechanisms, analyze the marginal gains of complex network structures on modeling accuracy in the massive state space of Tetris, and verify the adaptability of deep networks to the optimization framework proposed in this paper.

Code Availability

The source code for the bitboard-based Tetris AI is hosted on GitHub at: <https://github.com/GameAI-NJUPT/BitboardTetris>. All code is released under the MIT License, ensuring free access, modification, and distribution for academic and non-commercial purposes.

Acknowledgements

This study was supported by the National Natural Science Foundation, China (Nos. 62276142, 62206133, 62202240, and 62506172).

References

1. Demaine ED, Hohenberger S and Liben-Nowell D. Tetris is hard, even to approximate. In *Computing and Combinatorics: 9th Annual International Conference, COCOON 2003 Big Sky, MT, USA, July 25–28, 2003 Proceedings 9*. Springer, pp. 351–363.
2. Thiery C and Scherrer B. Building controllers for tetris. *Icga Journal* 2009; 32(1): 3–11.
3. Burgiel H. How to lose at tetris. *The Mathematical Gazette* 1997; 81(491): 194–200.
4. Brzustowski J. *Can you win at Tetris?* PhD Thesis, University of British Columbia, 1992.
5. Whiteson S, Tanner B and White A. The reinforcement learning competitions. *AI Mag* 2010; 31(2): 81–94.
6. Gabillon V, Ghavamzadeh M and Scherrer B. Approximate dynamic programming finally performs well in the game of tetris. In *Advances in neural information processing systems*. p. 1754–1762.
7. Scherrer B, Ghavamzadeh M, Gabillon V et al. Approximate modified policy iteration and its application to the game of tetris. *J Mach Learn Res* 2015; 16(49): 1629–1676.
8. Elfving S, Uchibe E and Doya K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks* 2018; 107: 3–11.
9. Lichtenberg JM and Şimşek Ö. Regularization in directable environments with application to tetris. In *International Conference on Machine Learning*. PMLR, pp. 3953–3962.
10. Szita I and Lörincz A. Learning tetris using the noisy cross-entropy method. *Neural computation* 2006; 18(12): 2936–2941.
11. Kauten C. Tetris (NES) for OpenAI Gym. GitHub, 2019. URL <https://github.com/Kautenja/gym-tetris>.
12. Bertsekas D and Tsitsiklis JN. *Neuro-dynamic programming*. 1 ed. Athena Scientific, 1996.
13. Nelson KE and Scherer MK. Jpytype. Technical report, Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), 2020.
14. Fahey C. Tetris ai, computer plays tetris, 2003.
15. Thiery C and Scherrer B. Improvements on learning tetris with cross entropy. *Icga Journal* 2009; 32(1): 23–33.
16. Amundsen JB. *A comparison of feature functions for Tetris strategies*. Master's Thesis, Norwegian University of Science and Technology, 2014.
17. Schulman J, Wolski F, Dhariwal P et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:170706347* 2017; .
18. Schulman J, Levine S, Abbeel P et al. Trust region policy optimization. In *International conference on machine learning*. pp. 1889–1897.